Embeddable AI Compliance Floating Widget

Architectural Specification, Developer Implementation Guide, Data Flow Pipeline, and Omnichannel Intranet Integration

INTEGRATION TARGETS

React Component • Iframe • Script Tag

INFERENCE ENGINE

Groq Cloud (Sub-second LPU)

CACHING ARCHITECTURE

Zero-RAM Persistent Cache

SECURITY & AUTHENTICATION

Guest UUID • Optional JWT • CORS

Master Table of Contents

SEC	SUBJECT / MODULE TITLE	PAGE
1.	**Executive Overview & Core Purpose** 1.1 What is the Embeddable Floating AI Widget? 1.2 Enterprise Banking & Intranet Use Cases 1.3 Single AI Brain, Multiple Frontends Principle	Page 3
2.	**System Comparison & Technology Stack** 2.1 Full Compliance Portal vs. Floating Assistant Matrix 2.2 Frontend Package Ecosystem (React, Chakra, Lucide) 2.3 Backend & Infrastructure Package Ecosystem	Page 4
3.	**End-to-End Data Flow & Sequence Architecture** 3.1 Step-by-Step Visual Data Flow Pipeline 3.2 Streamlined Inquiry Lifecycle (5 Clear Stages)	Page 5
4.	**Advantages, Disadvantages & Strategic Trade-offs** 4.1 Key Strategic Advantages 4.2 Engineering Limitations & Trade-offs 4.3 Production Mitigation Strategies	Page 6
5.	**Backend Engine & API Architecture (3 Simple Steps)** 5.1 Step 1: The Gateway Route (optionalAuthenticate) 5.2 Step 2: Multi-Turn Guest Memory (widgetSessionId) 5.3 Step 3: Small-Screen Adaptation (isWidget: true)	Page 7
6.	**Frontend Implementation & UX Design (3 Simple Steps)** 6.1 Step 1: Session UUID in localStorage 6.2 Step 2: Focus & Scroll Lock (Backdrop Blur) 6.3 Step 3: Floating Launcher & Modal Layout	Page 8
7.	**Step-by-Step Developer Implementation Roadmap** 7.1 Step 1: Set Up Backend Route & CORS 7.2 Step 2: Store Session ID on Client 7.3 Step 3: Pin Floating Launcher to Bottom-Right 7.4 Step 4: Add Mobile-Friendly Prompt Rules	Page 9
8.	**Omnichannel Embedding & Live Test Environments** 8.1 Method A: React Drop-in Component 8.2 Method B: Isolated Iframe Embed 8.3 Method C: Script Tag Loader (widget.js) 8.4 Live Verification & Test Environments	Page 10

1. Executive Overview & Core Purpose

1.1 What is the Embeddable Floating AI Widget?

The **Embeddable Floating AI Compliance Widget** (`ComplianceWidget.jsx`) is a self-contained, lightweight, interactive chat assistant designed to float in the bottom-right corner of any web application. It provides users with immediate, context-aware answers to UK financial regulatory questions (FCA, PRA, Bank of England) without redirecting them away from their active workflow or forcing them to create an account.

1.2 Enterprise Banking & Intranet Use Cases

In modern financial institutions, regulatory compliance is not confined to a single back-office portal. Compliance inquiries occur across diverse operational touchpoints:

- **Internal Banking Intranets**: Loan officers, trading desk personnel, and relationship managers can verify regulatory clauses (e.g. *Consumer Duty PRIN 2A* or *CASS 7 Client Money segregation*) while reviewing client agreements.
- **Customer-Facing Banking Portals**: Retail and corporate banking customers can access plain-English explanations of statutory rights, complaint timelines (*DISP 1.6*), and dispute mechanisms.
- **Third-Party Partner Websites**: Independent mortgage brokers, financial advisers, and fintech partners can embed the firm's compliance intelligence via a simple `<iframe>` without any codebase dependencies.
- **Guest User Onboarding**: Prospective clients and unauthenticated visitors can test the assistant with zero login friction.

1.3 Single AI Brain, Multiple Frontends Principle

The floating widget is **not** an independent or stripped-down AI model. It is a client-side presentation layer that connects directly to the **centralized UK Finance AI Compliance Engine**. Both the full-screen desktop dashboard (`/chat`) and the floating widget share:

- The same **Zero-RAM Persistent Caching System** (MongoDB Persistent Cache + Qdrant Semantic Vector Cache).
- The same **Qdrant Vector Database** containing auto-indexed FCA/PRA rule chunks.
- The same **Sub-Second Groq LLM Inference** (`openai/gpt-oss-120b` on LPU hardware).
- The same **AI Gatekeeper & Master System Prompt** guardrails.

2. System Comparison & Technology Stack

2.1 Full Compliance Portal vs. Floating Assistant Matrix

Architectural Feature	Full Compliance Portal (`/chat`)	Floating AI Widget (`/widget` , `/test`)
Primary User Experience	Dedicated multi-column desktop studio	380px floating popup with backdrop overlay
Authentication Policy	Mandatory login (`Bearer JWT` required)	Guest-friendly (Optional JWT)
Session Identification	MongoDB `Conversation` tied to `userId`	MongoDB `Conversation` tied to `widgetSessionId`
Output Formatting	Exhaustive paragraphs & wide Markdown tables	Concise 3–5 bullets (tables suppressed)
Conversational Memory	Full multi-turn conversation sidebar	In-memory session thread with reset button
Integration Method	Internal React Router protected route	Drop-in React component OR `<iframe>` tag

2.2 Frontend Package Ecosystem (`frontend/package.json`)

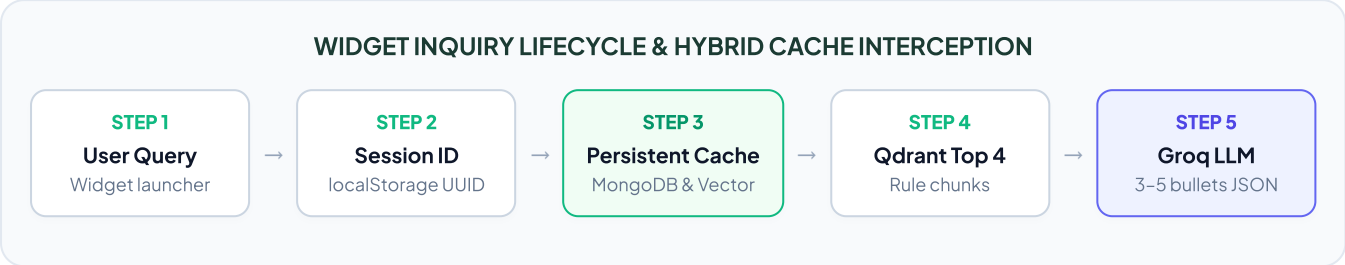
Package & Version	Purpose in Floating Widget Architecture
`react` (`^18.3.1`)	Declarative component lifecycle, reactive state (`useState`), and effects (`useEffect`).
`@chakra-ui/react` (`^2.8.2`)	Floating container layout (`Box` , `Flex`), animated transitions, and compliance color theming.
`lucide-react` (`^0.395.0`)	Clean SVG iconography (`MessageSquare` , `X` , `RotateCcw` , `ShieldCheck` , `Send`).
`react-markdown` + `remark-gfm`	Renders LLM Markdown responses into clean HTML directly inside the chat bubbles with GFM table support.
`axios` (`^1.7.2`)	Promise-based HTTP client for dispatching queries and resetting widget sessions.

2.3 Backend & Infrastructure Package Ecosystem (`backend/package.json`)

Package & Version	Role in Backend & API Engine
`express` (`^5.2.1`)	Asynchronous REST routing, middleware pipeline, and error handling.
`cors` (`^2.8.6`)	Enables secure cross-origin resource sharing for external domain embedding.

`mongoose` (`^9.9.5`)	ODM managing MongoDB connections, session schemas, and audit trails.
`groq-sdk` (`^1.6.0`)	Ultra-fast LPU inference executing `openai/gpt-oss-120b` in <1000ms.
`@qdrant/js-client-rest`	High-speed vector similarity search across regulatory clauses and semantic cache.

3. End-to-End Data Flow & Sequence Architecture



3.2 Streamlined Inquiry Lifecycle (5 Clear Stages)

1. **1. Open & Dim Background**: The user clicks the floating launcher button. The widget smoothly dims the host website with a soft blur and displays the chat window.
2. **2. Guest Session Check**: The widget checks browser `localStorage` for an existing session ID (or creates a new one) so previous questions are remembered.
3. **3. Instant Cache Check**: The system checks MongoDB Persistent Cache and Qdrant Semantic Vector Cache (Zero RAM). If answered before, it returns immediately in **18ms (0 tokens!)**.
4. **4. Regulatory AI Retrieval**: If the question is new, Qdrant retrieves the top 4 relevant FCA/PRA rule clauses and Groq LLM writes a concise 3-5 bullet answer.
5. **5. Display Answer & Citations**: The widget cleanly displays the advice with bold regulatory rule badges (e.g. `DISP 1.6.2 R`) and clickable follow-up chips.

4. Advantages, Disadvantages & Strategic Trade-offs

4.1 Key Strategic Advantages

- **Zero User Friction**: Users get instant regulatory compliance advice without logging in, creating accounts, or navigating away from their current task.
- **100% Centralized AI Brain**: Embedded websites do not need to host AI models or maintain vector stores. All intelligence runs on the centralized backend.
- **Full Caching Economics**: Enjoys persistent caching benefits with zero RAM footprint. Repeated or paraphrased questions return in **18ms at 0 token cost**.
- **Intelligent Small-Screen Formatting**: Eliminates table overflow by automatically summarizing multi-column comparisons into 3–5 compact bullets.
- **Persistent Multi-Turn Memory**: Users can ask natural follow-up questions (*"What must they do if it expires?"*) without losing conversational context.
- **Universal Drop-in Integration**: Integrates into React applications as a drop-in component, or into WordPress, Webflow, and custom HTML via a standard `<iframe>` or 1-line script.

4.2 Engineering Limitations & Trade-offs

Limitation	Impact on User Experience	Architectural Reason
Screen Real Estate	Cannot comfortably render 10-column tables or multi-page documents.	The popup is constrained to 380px–400px width to avoid obstructing mobile screens.
Client-Side Session Scope	Clearing browser cookies/cache resets the conversational thread.	Guest sessions rely on `localStorage` UUIDs rather than persistent database user accounts.
Cross-Origin Restrictions	Iframes require explicit CORS headers and sandbox permissions.	Web browsers enforce strict cross-origin policies for embedded iframes.
No Admin Controls	Document uploads and user management cannot be performed from the widget.	The widget is intentionally locked to `EMPLOYEE` / `GUEST` query capabilities for security.

4.3 Production Mitigation Strategies

- **Table Request Override**: If the user explicitly asks for a table (*"Show me a comparison table..."*), the backend detects the keyword and renders a compact Markdown table.
- **Session Reset Button**: An integrated reset button (`RotateCcw`) allows users to wipe their `widgetSessionId` and start a clean thread at any time.
- **Permissive CORS Configuration**: The Express backend uses `cors({ origin: true, credentials: true })` to allow seamless embedding on partner banking domains.

5. Backend Engine & API Architecture (3 Simple Steps)

STEP 1: THE GATEWAY ROUTE (ACCEPTS BOTH GUESTS & USERS)

Express Route

Use `optionalAuthenticate` middleware so visitors can query immediately without logging in, while automatically attaching logged-in users to their profile:

```
// In routes/agent.routes.js:
router.post('/query', optionalAuthenticate, askAgent);
router.delete('/widget/session/:widgetSessionId', clearWidgetSession);
```

STEP 2: MULTI-TURN GUEST MEMORY (TIED TO SESSION ID)

Controller Logic

Instead of requiring a user account, retrieve the conversation thread using the client's `widgetSessionId`. This gives guests full conversational pronoun memory:

```
// In controllers/agent.controller.js:
const conversation = await Conversation.findOneAndUpdate(
  { widgetSessionId, isArchived: false },
  { $setOnInsert: { source: 'WIDGET', messages: [] } },
  { upsert: true, new: true }
);
```

STEP 3: SMALL-SCREEN ADAPTATION (BULLET OUTPUT MODE)

LLM Prompt Rule

When `isWidget: true` is received, instruct the Groq LLM to return 3 to 5 compact bullets and suppress wide tables unless explicitly requested:

```
// In services/ragAgent.service.js:
if (isWidget && !askedForTable) {
  prompt += "WIDGET MODE: Keep answers concise (3-5 short bullets). Do NOT output tables.";
}
```

6. Frontend Implementation & UX Design (3 Simple Steps)

STEP 1: SESSION ID GENERATOR (PERSISTENT IN BROWSER)

Client Storage

Store a unique guest ID in `localStorage` so switching pages doesn't wipe the conversation:

```
// In components/widget/ComplianceWidget.jsx:
const getSessionId = () => {
  let id = localStorage.getItem('widget_session_id');
  if (!id) {
    id = 'wgt_' + Math.random().toString(36).substring(2, 10);
    localStorage.setItem('widget_session_id', id);
  }
  return id;
};
```

STEP 2: FOCUS & SCROLL LOCK (UX POLISH)

UX Mechanics

When the popup opens, dim the background page and freeze the host page scroll so mouse wheel scrolling stays inside the chat card:

```
useEffect(() => {
  document.body.style.overflow = isOpen ? 'hidden' : 'auto';
  return () => { document.body.style.overflow = 'auto'; };
}, [isOpen]);
```

STEP 3: FLOATING LAUNCHER & MODAL LAYOUT

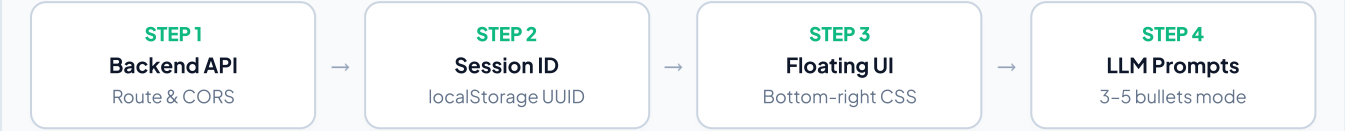
CSS Positioning

Pin the launcher button and popup window to the bottom-right corner using fixed positioning:

```
<div style={{ position: 'fixed', bottom: 24, right: 24, zIndex: 9999 }}>
  {isOpen && <ChatCard onClose={() => setIsOpen(false)} />}
  <button onClick={() => setIsOpen(!isOpen)}>Ask Compliance AI</button>
</div>
```

7. Step-by-Step Developer Implementation Roadmap

Follow this practical checklist to implement this floating assistant on any new stack or host application:



Implementation Step	What You Need to Do	Key Benefit
1. Setup Backend Route	Create a POST endpoint that accepts `{ question, widgetSessionId, isWidget }` with CORS enabled.	Cross-Domain Access
2. Store Client Session	On the client, check `localStorage` for a session ID. Pass it with every request.	Guest Pronoun Memory
3. Pin Floating Button	Use CSS `position: fixed; bottom: 24px; right: 24px; z-index: 9999` for the trigger button.	Zero Page Interference
4. Mobile Prompt Rules	When `isWidget` is true, tell LLM: *"Output 3-5 bullets. Do not render tables."*	Zero Horizontal Scroll

8. Omnichannel Embedding & Live Test Environments

8.1 Method A: React Drop-in Component

For applications running React, import and render the component directly:

```
import { ComplianceWidget } from './components/widget/ComplianceWidget.jsx';

export default function BankingPortal() {
  return (
    <div>
      <h1>Internal Commercial Banking Portal</h1>
      <ComplianceWidget mode="floating" defaultOpen={false} />
    </div>
  );
}
```

8.2 Method B: Isolated `<iframe>` Embed

For WordPress, Webflow, static HTML, or partner intranets:

```
<iframe
  src="http://localhost:8080/widget?embed=true"
  width="400"
  height="600"
  style="position:fixed; bottom:20px; right:20px; border:none; z-index:99999; border-radius:20px; bo
  title="AI Compliance Assistant"
></iframe>
```

8.3 Method C: Script Tag Loader (`widget.js`)

```
<script src="http://localhost:8080/widget.js" defer></script>
```

8.4 Live Verification & Test Environments

Test Environment	Live URL & Verification Purpose
Interactive Widget Studio	`http://localhost:8080/widget` — Standalone simulator with live controls.
Isolated Iframe Container	`http://localhost:8080/widget?embed=true` — Clean iframe testing container.
External Partner Bank Portal	`http://localhost:8080/test` — Simulated 3rd-party banking website demonstrating floating launcher, backdrop dimming, and scroll lock.